



Touching Loops and Worms Formed by Cellular Automata

Rolf Hoffmann^(✉) 

Technical University Darmstadt, Darmstadt, Germany
hoffmann@rgb.informatik.tu-darmstadt.de

Abstract. The objective is to form patterns with “touching” loops by Cellular Automata (CA). A loop is defined as a closed path of 1-cells in a 2D grid on a zero background and with a zero border. A path cell has two adjacent path cells, and two path cells are allowed to touch each other on a diagonal. The designed CA rule tries to build loops by overlapping tiles. It uses so-called *templates*, 5×5 matching patterns which are systematically derived from the problem specific defined set of 3×3 tiles. The rule checks the pattern being evolved against the list of templates. If the outer neighbors of a template match, then the cell’s state is set to the template’s center value. Noise is injected if there is no matching template or certain constraints are not fulfilled. “Worms” are special loops which have a maximum of inner self-touching points. The designed Rule 1 variant evolves stable patterns with any kind of loop structures in a short time, whereas the Rule 2 variant evolves stable worm-only patterns in a much longer time. For different grid sizes, a selection of generated loop patterns are presented and some of their properties are discussed.

Keywords: Pattern Formation · Evolving Loop Structures · Overlapping Tiles · Matching Templates · Probabilistic Cellular Automata · Asynchronous Updating

1 Introduction

Loops are common structures that appear in a natural way or by design and construction. They can be found in various organisms (e.g. closed blood circulation system, veins in tree leaves), physical systems (e.g. topological magnetic structures like Hopfions [1] and geophysical systems (highly porous limestone, cracks on the surface of glazed ceramics), chemical systems (e.g. cyclic polymers [2], cyclic nanostructures [3]), biological systems (e.g. [4, 5]), or also in the products of human activity (e.g. street networks in cities). Such reticulated patterns (as opposed to tree patterns, where loops are absent) usually arise as a result of complex interactions, and understanding the mechanisms leading to their formation are a fascinating and challenging subject of research. On the other hand, patterns containing loops can be the subject of research in the field of designing algorithms leading to their creation.

The Objective. We consider 2D grid patterns of size $n \times n$ where a cell has the state value/color $s \in \{0, 1\}$. The aim is to find a CA rule that can form patterns with loops. We allow that a loop can touch itself (*self-touching*), and that individual loops can touch each other. In addition we aim at so-called “worm patterns”. Worms are special densely contracted self-touching loops. The main challenge was how to form a loop (its path and closing it) by local conditions. The solution presented is the definition of a set of tiles (local patterns acting as local conditions) that are allowed and forced to overlap (aggregate, self-assemble) taking into account certain constraints.

Related Work. Probabilistic CA rules using overlapping tiles can generate point patterns [7], domino patterns [8], and sensor point patterns [9]. In [10] loops were formed under cyclic boundary conditions, first with a Genetic Algorithm and then with a CA rule also based on overlapping tiles.

We will also follow the idea of using a probabilistic rule and overlapping tiles. In contrast to the cited work, here (i) a different and smaller set of tiles was designed in order to evolve loops that can touch, (ii) the boundaries were fixed which is more challenging than using cyclic boundary conditions, and (iii) an additional constraint was defined allowing to evolve worm-only loops.

Overlapping 1D tilings are treated in [6], in the context of recognizable languages, monoids, and two-way automata, and this paper could be a good basis for further studies and extensions for the n -dimensional case.

Our problem is related to space-filling curves and Hamiltonian cycles [11, 12]. Generating such curves by a CA rule is a task that could be addressed. For space-filling curves, the field size has to be considered with respect to the possible solutions.

There are several publications on “Peterson’s worms” and their derivatives understood as moving agents which may form cyclic paths under the constraint that every grid connection can be “eaten” only once, e.g. [13]. In contrast, here the loops evolve asynchronously cell by cell without using an agent and a sophisticated control algorithm. In other work (e.g. [14]) worms are complex entities composed of several cells that are able to move around. It is interesting that loop structures may appear in the *Game of Life* as “Still Life”, but they appear very rarely among many other stable objects [15].

Paper Structure. In Sect. 2, the properties of loops and loop patterns are explained. The design method using defined tiles and templates is presented in Sect. 3. In Sect. 4, the designed CA rule (in two variants) is defined. The loop patterns evolved by Rule 1 are discussed in Sect. 5.1, and worm-only patterns evolved by Rule 2 are shown in Sect. 5.2. Conclusions and future work finalize the paper.

2 General Aspects of Loops and Loop Patterns

A *loop* is modeled as a *cyclically closed path of loop path cells* (1-cells) embedded in a grid of size $n \times n$ with a zero background and a zero boundary. A *loop pattern*

is a pattern which contains at least one loop. A *loop path cell* is a 1-cell that has exactly two 1-cells in NESW, where NESW are the four orthogonal adjacent von-Neumann neighbors. Formally the following condition (so-called *loop path condition*) is true for all path cells of a loop:

$$(s = 1) \wedge ((s^N + s^E + s^S + s^W) = 2)$$

where the cell's main state is $s = 1$ for path cells and $s = 0$ for the surrounding cells. The term $s^N + s^E + s^S + s^W$ gives the number of NESW-neighbors which are in state $s = 1$. The loop path condition ensures that three path cells in sequence form a horizontal straight line segment $\begin{smallmatrix} & 0 \\ 1 & 1 & 1 \end{smallmatrix}$ (or a vertical one, resp.), or a corner $\begin{smallmatrix} 0 & 0 \\ 0 & 1 & 1 \\ & 0 & 0 \end{smallmatrix}$ (4 variants by rotation). In order to form a loop it is required that the neighboring path cells of a path cell also fulfill the loop path condition, and in this way all path cells in continuation will form a cyclically closed path.

The loop path condition also allows corner path cells (more precisely the heads/outer corner points of corners) to touch each other on a diagonal, i.e. either the two path cells (*NorthEast*, *SouthWest*) or (*NorthWest*, *SouthEast*) can be neighbors. The outer two path cells belonging to a corner lie also on a diagonal but they are not considered as touching. So touching path cells fulfill the local patterns $\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}$ or $\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}$.

The loop's path shall be surrounded by 0-cells forming a *hull*. This requirement is already guaranteed by the loop path condition, with the exception that a cell diagonally located outside of a corner can be 0 (background) or 1 (another touching path cell). The hull ensures that loop paths keep a minimal distance between each other, and also within a loop itself. Hull cells are allowed to be used by several paths, from the same or other loops, i.e. hull cells may overlap.

For reasoning about loop attributes we can imagine that an agent walks clockwise in one-way around the loop until it returns back to its starting point. The agent follows its path by going stepwise straight, to the right or to the left. Attributes of loops and loop patterns are:

- $P \in \{0, 1\}^{n \times n}$ – pattern.
- $N_{Loop}(P)$ – number of loops in the pattern P .
- $\lambda(i)$ – length of loop with index i , the number of its path cells.

The loop indexes $i \geq 1$ are assigned by scanning the pattern row by row from top to bottom, and from left to right in each row.

- $\alpha(i)$ – (enclosed area) number of cells which loop i encloses.
- $\delta(P) = N_{ones}(P)/(n+1)^2$ – density of 1-cells in a pattern P .

where $N_{ones}(P)$ is the number of ones in the pattern. The divisor $(n+1)^2$ is used instead of n^2 because then the maximal possible density becomes 0.5. This is the case for instance for space filling curves. So we include half of the border cells into the number of all cells being considered.

- $R(i) \geq 4$ – number of right turns for a circuit-making agent in loop i . A right turn corresponds to a (convex) outer corner pointing to the loop's outside.

- $L(i) \geq 0$ – number of left turns in loop i . A left turn corresponds to a (concave) inner corner pointing to the loop's inside. The following assertion is true:

$R - L = 4$, or $L = \Delta R$ where $R = 4 + \Delta R$, and '4' corresponds to the 360° turn for making a circuit.

- $b^{NESW}(i) = (b^N, b^E, b^S, b^W)(i)$ – number of loop i 's path cells that are adjacent to the N(orth)/E(ast)/S(outh)/W(est) border.

Let us look at the patterns depicted in Fig. 1(a–d). Pattern (a) contains two nested loops, loop 1 encloses loop 2 (the 3×3 so-called mini-loop), and its values are

$$\lambda(1) = 4 \times 6 = 24; \quad \lambda(2) = 8; \quad R(1) = R(2) = 4; \quad L(1) = L(2) = 0; \\ \delta = 32/8^2 = 0.5; \quad b^{NESW}(1) = (7, 7, 7, 7); \quad b^{NESW}(2) = (0, 0, 0, 0).$$

The pattern in Fig. 1(b) has the values

$$\lambda = 24; \quad R = 8; \quad L = \Delta R = 4; \quad b^{NESW} = (3, 3, 3, 3).$$

The pattern in Fig. 1(c) has the values

$$\lambda = 26; \quad \delta = 26/64 = 0.406; \quad R = 7; \quad L = 3; \quad b^{NESW} = (6, 4, 3, 7).$$

Figure 1(d) is a space-filling Hamiltonian circuit with the values

$$\lambda = 32, \delta = 32/64 = 0.5; \quad \alpha = 13; \quad R = 8; \quad L = 4.$$

Figure 1(i) is a so-called “max-worm” with the values

$$\lambda = 24; \quad \delta = 0.375; \quad \alpha = 5; \quad R = 12; \quad L = 8; \quad b^{NESW} = (3, 3, 3, 3).$$

Touching Points. We can distinguish four different types of touching points:

- *self-touching point*
 - *inner*: Two cells are touching with the *inner* side of the loop path. Such a point is called “*inner self-touching point*” (Fig. 1(e)).

- *outer*: Two cells are touching with the *outer* side of the loop path. Such a point is called “*outer self-touching point*” (Fig. 1(f)). Such points are bottlenecks within caves or bottles with an entry from outside the loop.
- *inter-touching point*
 - *external*: Two cells of different loops A and B are touching. If both outer sides of the loops’ paths are touching, then the point is called “*external inter-touching point*” (Fig. 1(g)). This is the more frequent case when loop A is situated beside loop B.
 - *internal*: Two cells of different loops A and B are touching where A encloses B. Then the inner side of path A touches the outer side of path B. Such a point is called “*internal inter-touching point*”, or “*enclosing inter-touching point*” (Fig. 1(h)).

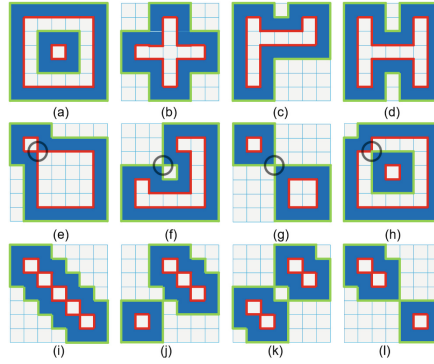


Fig. 1. Some specific 7×7 loop patterns. (a) Two nested loops. (b) Loop with 4/4 outer/inner corners, and (c) with 5/3. (d) A space-filling loop. (e) Inner self-touching, (f) outer self-touching, (g) external inter-touching, (h) internal inter-touching loop. (i) Worm of maximal size 4, (j) 2-worm inter-touching with a mini-loop of size 3×3 , (k) two 1-worms, (l) 1-worm with mini-loop. Path cells in blue, inner/outer side of a loop’s path in red/green. (Color figure online)

Worms. We define a “*worm*” as a loop that is strongly contracted with a maximal number of inner self-touching points, and it forms a diagonal of zeroes enclosed by ones. We do not consider the simple square 3×3 loop (mini-loop) as a worm because it has no touching point. A k -worm is a worm that has $k \geq 1$ inner self-touching points. The number of enclosed zeroes is $\alpha = k + 1$ which corresponds to the length of the zero-diagonal. The worm’s length is $\lambda = 4(k + 2) = 4(\alpha + 1)$. A *max-worm* is the largest worm that can fit into a square grid of size n^2 , which is a $(k = n - 3)$ -worm. Some 7×7 patterns with worms are shown in Fig. 1(i – l), (i) shows a max-worm with the maximum of 4 inner self-touching points, (j) a 2-worm inter-touching with a mini-loop, (k) two 1-worms, and (l) a 1-worm inter-touching with a mini-loop.

3 The Design Method Using Tiles and Templates

The rule was designed on the basis of a method already applied successfully for evolving point patterns [7], domino patterns [8], sensor networks [9], and loop patterns [10]. The steps of the updated method are:

1. A *set of tiles* has to be defined that can induce the desired pattern. A *tile* is a small pattern consisting of elementary tiles (square cells of minimal size according to the grid) which are called *pixels* in order to distinguish them from the *cells* of the aimed pattern. The defined tiles (and replicates of them) are then used to fill the given space. Pixels from different tiles are allowed to overlap if there is no conflict in color. If K pixels of the same color are overlaid, then the cover level v of this cell becomes $v = K$. Depending on the set of tiles and the given space it is not always possible to cover the whole space ($v > 0$ for all cells) if that is required.
2. So-called *templates* are derived from the tiles. For each pixel of a tile a template is derived by shifting the tile in such a way that the chosen pixel becomes the center. This means that for every tile pixel a specific template is generated.
3. The set of all templates is inserted into a general probabilistic CA rule, and the rule is applied in the following way:
 - All outer templates (the templates without the center) are tested against the actual cell's neighborhood. If there are hits, they are registered together with their templates.
 - If there is only one hit, the cell's value is adjusted to the center value of the hitting template. If there are several hits, then one of the hitting templates is chosen at random for adjustment.
 - If there is no hit, then noise is injected. This means that the cell's state is changed into another state with a certain probability. In addition, noise can be injected if certain constraints are not fulfilled.

3.1 The Defined Set of Tiles

For the construction of touching loops only two basic (proto-) tiles are necessary:

(a) a *straight path tile* $\begin{smallmatrix} 1 & 0 & 1 \\ & 0 & \\ 0 & 1 & 0 \end{smallmatrix}$ responsible for the straight continuation of a path, and (b) a *corner tile* $\begin{smallmatrix} 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{smallmatrix}$. Each tile contains three adjacent one-pixels. In order to yield the full set of tiles the symmetric ones have to be added (Fig. 2), because we aim at patterns which do not give preference to a certain direction.

3.2 The Derived Templates

What is a *template*? A template is a local matching pattern. It defines local conditions/constraints for valid patterns. For each tile T of the tile set, K templates $T_0, T_1, \dots, T_{K-1}$ are derived, where K is the number of valid tile pixels with the

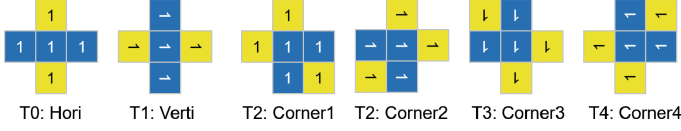


Fig. 2. The whole set of tiles, including the symmetric ones. The added numbers in the tile pixels indicate the cover/overlay level, ‘1’ means that they are not overlaid.

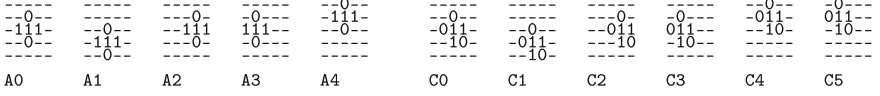


Fig. 3. The five templates $A0 - A4$ derived from tile $A0 = \text{Hori}$, and the six templates $C0 - C5$ derived from tile $C0 = \text{Corner1}$. The symbol ‘-’ denotes sites which are don’t cares where no valid pixel exists and where no test is performed.

value 0 or 1. The template T_i is derived by shifting the tile in way that the valid pixel i becomes the center of that template. By convention we define $T_0 = T$, i.e. the template T_0 is equal to the tile without shifting. Note that we assume a center/anchor for each tile, and also for each template. Figure 3 shows (i) the five templates $A0 - A4$ derived from tile Hori , where $A0 = \text{Hori}$ and $A1 = \text{ShiftDown}(A0)$, $A2 = \text{ShiftRight}(A0)$, $A3 = \text{ShiftLeft}(A0)$, $A4 = \text{ShiftUp}(A0)$, and (ii) the six templates $C0 - C5$ derived from tile Corner1 , where $C0 = \text{Corner1}$. Then, in the same way, the templates for the other tiles are derived. The number of all templates becomes $2 \times 5 + 4 \times 6 = 34$, a template for each tile (2 + 4) and for each pixel.

4 The CA Rule

Cell State. The used CA cell state at a site (x, y) is $q = (s, h_0, h_1)$, where $s \in \{0, 1\}$ is the main pattern state (color), and $h_0 \in \{0, 1, \dots, 8\}$ is a template hit with $s = 0$, and $h_1 \in \{0, 1, 2, 3\}$ is a template hit with $s = 1$, and (x, y) are the coordinates (indexes) of the cell. It can be noticed that the hit values converge to the cover levels when a pattern stabilizes.

Asynchronous Updating. We want to use *asynchronous* execution/simulation of the CA rule. For each generation (time-step $t \rightarrow t + 1$), n^2 cell operations (micro-steps $\tau \rightarrow \tau + 1$) are executed. A simple scheme is used with the following cell operation at each micro-step

1. A cell (x, y) is selected at random.
2. The new state value s' at (x, y) is computed by the CA rule (for details see next paragraph).

3. The new state value s' is immediately copied to the state variable $s(x, y)$ yielding the next state.

For output it is useful to compute also the *cover levels* for each considered configuration (displayed pattern). This can be done by testing at each site all tiles and storing the number of pixels that overlay.

Rule Details. The rule is composed of two function evaluations (steps) in sequence. In the first step a template test is performed. In the second step the state is corrected (adjusted) if there is a matching template. In the third step noise is injected if certain constraints are not satisfied. Note that the rule as a whole forms a function although it is computed stepwise.

1. TEST for template hits

All templates are tested against the current pattern at site (x, y) where the center of each template is aligned to this site. The test is performed only on the outer neighborhood without considering the central state $s(x, y)$. So the current value of the cell is not taken into account, only the real neighbors.

$(h_0, h_1, k) \leftarrow \text{ComputeHitsForAllTemplates } T_i \in \{T_0, T_1, \dots, T_{K-1}\}$

where

h_0 = number of zero-hits (hits with template center value = 0)

h_1 = number of one-hits (hits with template center value = 1)

k = index of a matching template T_k

$K = 34 = 4 \times 2 + 6 \times 4$, the number of templates (#valid pixels in all tiles).

2. ADJUST if hit

The default new state is $s' = s$ (keep old state). If there is a hit, the state is adjusted to the center of a matching template.

$s' = \text{center}(T_k) \quad \text{if } h = h_0 + h_1 > 0$

where

$\text{center}(T_k)$ is the value of the center pixel of a matching template T_k .

If there is only one hit, the cell's state s is adjusted to the value of the template's center value. This means that the center color is corrected if all other pixels of the template are correct. In the case when the whole template (including the center) correctly matches, the adjustment is not necessary, it is redundant. If there are several hits, one of the hitting templates is selected at random for adjustment.

3. NOISE INJECTION

The current new state (temporary variable) may be changed with a certain probability if the constraints are not fulfilled.

$$s' \leftarrow \begin{cases} \text{Random}(0/1) & \text{if } P \wedge (s' = 1) \wedge \text{TRUE}(\pi_1) \\ & \text{or } Q \wedge (s' = 0) \wedge \text{TRUE}(\pi_0) \\ & \text{or } \text{NOT}(W) \wedge (s' = 1) \wedge \text{TRUE}(\pi_2) \end{cases}$$

where

$\text{TRUE}(\pi) \Leftrightarrow \text{TRUE}$ with probability π else *FALSE*

$P = (h_1 \neq 3) \quad - \text{the 'loop path condition' for a closed loop}$

$Q = (h_0 = 0) \quad - \text{this constraint forbids uncovered 0-cells}$

$$W = (h_0^{North} = 6, 7) \vee (h_0^{East} = 6, 7) \vee (h_0^{South} = 6, 7) \vee (h_0^{West} = 6, 7) \\ - \text{the constraint for yielding worms}$$

Two rule variants were defined and simulated, **Rule 1** with the parameter settings $\pi_1 = 0.5, \pi_0 = 0, \pi_2 = 0$, and **Rule 2** (the *Worm-only Rule*) with the settings $\pi_1 = 0.5, \pi_0 = 0, \pi_2 = 0.5$. Rule 1 evolves loop patterns with all kinds of loops (non-touching, touching, worms), and Rule 2 patterns with worms only.

Both rules use the setting $\pi_2 = 0$ which means that empty uncovered areas are allowed in the pattern, and also the extreme case all-zero for the whole pattern. It turned out that this configuration appears very rarely (because its probability to appear becomes very low with an increasing grid size), and the evolution is faster compared to a setting $\pi_0 > 0$. Setting $\pi_0 > 0$ would exclude patterns with uncovered 0-cells (marked in red in the shown patterns). Then such interesting or demanded loop patterns like worm-only patterns with uncovered cells would be destroyed during the evolution and would appear only as transients, and most of the evolved patterns could appear very noisy. Therefore we used $\pi_0 = 0$. Nevertheless $\pi_0 > 0$ can be necessary if loop patterns without empty areas are required to be evolved by Rule 1. Further studies are required to determine the probability range of uncovered cells in the patterns for $\pi_0 = 0$.

5 Loop Patterns

5.1 Patterns Evolved with Rule 1

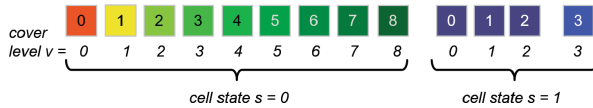


Fig. 4. The color scheme for the shown patterns, taking into account the cover level

Patterns of Size 7×7 . The pattern shown in the following use the color scheme as depicted in Fig. 4. Selected evolved 7×7 patterns are shown in Fig. 5. A loop encloses another in c5 and c6, and there are four inter-touching points in c6. Loops a6 and a7 have an entry-point (bottleneck) from outside the loop to a cave, there are cells which touch each other with the outside of the loop's path. Such outer-touching points can appear for $n \geq 7$. The patterns c4 – c7 are space-filling, the number of path cells is maximal (32). The loop c7 is a non-touching space-filling curve.

Patterns of Size $n = 8, 16, 32$. A selection of evolved 8×8 loop pattern is shown in Fig. 6. (I) Row (a) shows 8 loops with *one* self-touching point, and (b1 – c2) show 10 loops with *two* self-touching points. Loops (c3 – c6) have *three*

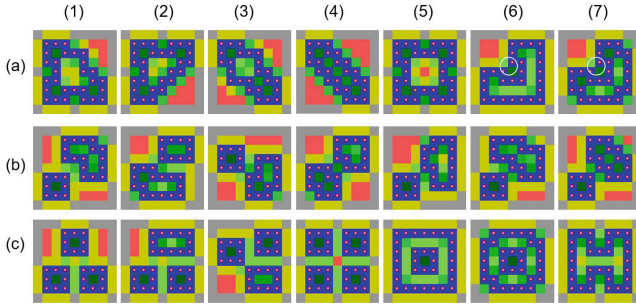


Fig. 5. A selection of 7×7 patterns. (a1, a2, a3) contain loops with 3 self-touching points, and (a4, a5) with 4. Loops a6 and a7 each have cells that touch each other from the outer side of the loop's path.

self-touching points, and loops (c7, c8) have *four*. Patterns (d1 – e1) have *two inter-touching* loops. Loops (e2, e3) each are loops that touch externally (with an *outer self-touching* point). (e5 – e8) are patterns with *worms*. The patterns (Fig. 6(II)) are seven 16×16 patterns with a different number of loops, and one 32×32 pattern with 5 loops.

The CA rule can evolve very many different, *stable* touching loop patterns. The rate of the evolved 8×8 patterns with one loop, with four (or more) was much lower than that for patterns with two or three loops.

Special regular patterns (like the ones shown in Fig. 5(c5 – c7) or Fig. 6(e1, e4 – e8) can appear, but the probability is low because the total amount of all valid loops patterns is very high. The evolution of a stable loop pattern with Rule 1 is relatively fast. After typically 10 generations partial loop structures appear in 8×8 patterns, but it takes more time ($t_{avg} = 51$ on average) to change all intermediate structures into stable closed loops.

Time-Complexity. The time-complexity was estimated by executing multiple runs and averaging. The number of performed runs (each yielding a stable pattern) was 100 for $n = 3, 4, \dots, 12$, and 50 for $n = 14, 16, 18, 20$, and 20 for $n = 24, 28$, and 10 for $n = 32$. Surprisingly, the time (time-steps t needed for a stable loop pattern) grows only approximately linear with the number of cells ($t = O(n^2)$), as we may conclude from the curve's trend (Fig. 7). This means that each cell's rule execution consumes a constant number of time-steps. (Here we got a special result where the constant is approx. 1.) In the parallel case, when a fraction α of the cells are updated in parallel, the parallel time could approximate a constant. To confirm this proposition more experiments and/or theoretical investigations are needed. One explanation for the low time-complexity could be that a large loop can evolve faster in an area that was restricted by smaller, already formed loops. The CA rule is able to evolve also larger patterns with $n > 16$ with a relatively low computational effort because of the low time complexity.

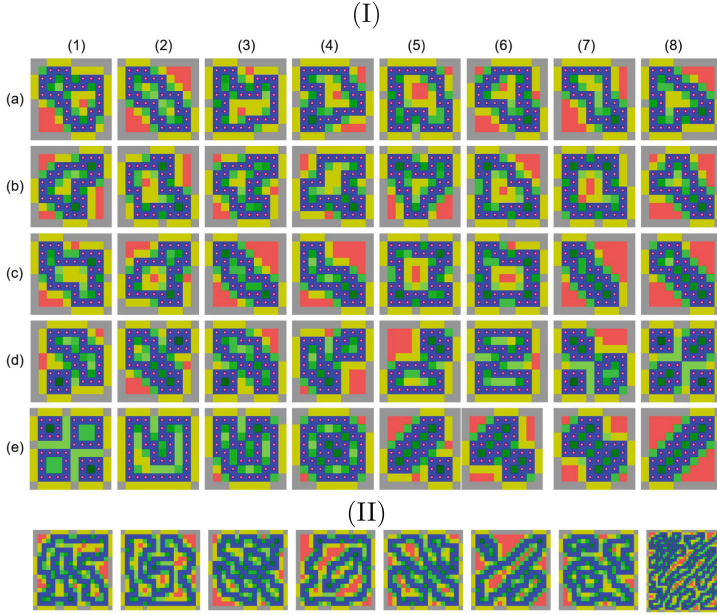


Fig. 6. (I) A selection of evolved 8×8 patterns, showing loops with (a) one self-touching point, (b1 – c2) two, (c3 – c6) three, and (c7, c8) four. Patterns (d1 – e1) contain two inter-touching loops. Loops e2 and e3 each contain an outer self-touching point, and (e5 – e8) are worm-only patterns.—(II) (1 – 7) are 16×16 patterns with (1, 1, 2, 2, 3, 3, 4) loops, (8) is a 32×32 pattern with 5 loops.

Density. The maximal reachable density depends on the grid size and the loop structure(s). It is $\delta_{max} = 0.5, 0.4, 0.44, 0.41, 0.5, 0.49$ for $n = 3, 4, 5, 6, 7, 8$. The upper bound is 0.5 which can only be reached for certain n (e.g. 3, 7). The density of the evolved patterns on average was

$$\delta_{avg} = 0.381, 0.363, 0.345, 0.349, 0.363, 0.379, 0.388, 0.393, 0.398, 0.402, 0.394, \\ 0.413, 0.414, 0.435, 0.435 \text{ for} \\ n = 5, 6, 7, 8, 9, 10, 11, 12, 4, 16, 18, 20, 24, 28, 32.$$

5.2 Worm-Only Patterns Evolved with Rule 2

The worm-only Rule 2 is more strict than Rule 1 because of the additional constraint W saying that path cells need zero-valued neighbors with cover level 6 or 7 in their Moore-neighborhood. Now worm-only patterns are the fixed points of the evolution. As the number of such patterns is much lower in comparison to the number of fixed points for Rule 1, the evolution takes much longer and is increasing super-linear in the number of cells. For example the average time to evolve always worm-only patterns for all used 60 runs was $t_{avg} = 207, 1223, 5262$

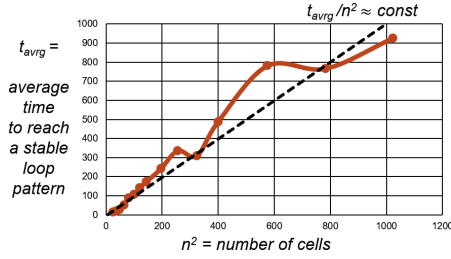


Fig. 7. In the simulation, the time-complexity is approximately $O(n^2)$. The data: $t_{avg} = 13, 19, 27.4, 51, 87.8, 106, 141.7, 172, 241, 335, 308, 487, 781, 766, 926$ for $n = 5, 6, 7, 8, 9, 10, 11, 12, 4, 16, 18, 20, 24, 28, 32$.

for $n = 4, 5, 6$, and the time needed per cell becomes $t_{avg}/n^2 = 13, 49, 146$ (1 : 3.8 : 11). Therefore a few of the shown patterns in Fig. 8 were constructed because they did not show up under the maximal time-limit setting of 100'000 generations. The patterns (b1, b6 – b8) were designed by hand and then used as initial configurations which stayed stable, therefore they can be evolved in principal. The shown patterns have the additional property that for each of the four boundaries there exists at least one adjacent path cell.

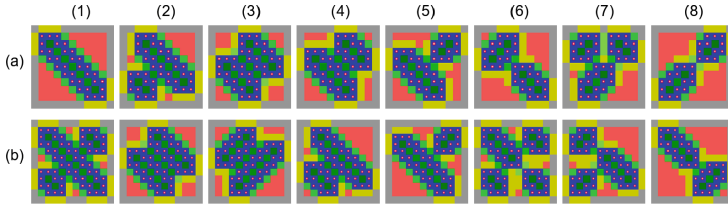


Fig. 8. A selection of worm-only patterns. (a) 9×9 patterns, and (b) 10×10 patterns.

6 Conclusions and Future Work

A CA rule was designed that can evolve stable touching loop patterns. The Rule 1 variant evolves all kind of loops, whereas the Rule 2 variant evolves worm-only patterns. First a set of 6 tiles was defined which are able to build paths in the grid. Each tile defines a 3×3 local pattern consisting of 0/1-pixels. 34 templates were derived from the tiles by shifting them in a way that each pixel becomes the center of a template. The templates were inserted into a probabilistic CA rule with asynchronous updating. All templates are tested for matching in the neighborhood of each CA cell. The rule tries to establish a template at each site by correcting the cell's value to the center value of a hitting template or by noise injection otherwise. In addition, the loop path conditions needs to be fulfilled,

saying that each path cell shall have an overlap level of 3. For Rule 1 the time-complexity was estimated by simulations, it depends only linearly in the number of cells. For Rule 2 the time-complexity increases super-linear, shown for some cases.—There are several topics for future work, like

- Larger spaces should be investigated in search of specific phenomena.
- How are the loops structured under cyclic boundary conditions?
- How can certain pattern characteristics like the number of loops, the loops length, or the density be maximized or minimized in a local way?
- How can the time-complexity be computed theoretically?
- How can worm-only patterns be evolved with less time, e.g. by using other tiles and constraints?
- Loops with special connections and intersections could be considered.
- How resistant are loop patterns against perturbations?
- The generation of Hamiltonian cycles could be addressed.

References

1. Kent, N.A.: Structure and Topology in Nanoscale Magnetic Systems. Doctoral dissertation. University of California, Santa Cruz (2020)
2. Kricheldorf, H.R.: Cyclic polymers: synthetic strategies and physical properties. *J. Polym. Sci. Part A: Polym. Chem.* **48**(2), 251–284 (2010)
3. Olenyuk, B., Fechtenkötter, A., Stang, P.J.: Molecular architecture of cyclic nanostructures: use of co-ordination chemistry in the building of supermolecules with predefined geometric shapes. *J. Chem. Soc. Dalton Trans.* **11**, 1707–1728 (1998)
4. Cardote, T.A., Ciulli, A.: Cyclic and macrocyclic peptides as chemical tools to recognise protein surfaces and probe protein-protein interactions. *ChemMedChem* **11**(8), 787–794 (2016)
5. Wang, C.K., Kaas, Q., Chiche, L., Craik, D.J.: CyBase: a database of cyclic protein sequences and structures, with applications in protein discovery and engineering. *Nucleic Acids Res.* **36**(suppl 1), D206–D210 (2007)
6. Janin, D.: On languages of one-dimensional overlapping tiles. In: van Emde Boas, P., Groen, F., Italiano, G.F., Nawrocki, J., Sack, H. (eds.) *SOFSEM 2013. LNCS*, vol. 7741, pp. 244–256. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-35843-2_22
7. Hoffmann, R.: Forming Point Patterns by a Probabilistic Cellular Automata Rule. [arXiv:2202.06656](https://arxiv.org/abs/2202.06656). Based on a *Presentation at Summer Solstice Conference on Complex Systems (June 2019)* (2022)
8. Hoffmann, R., Désérable, D., Seredyński, F.: A cellular automata rule placing a maximal number of dominoes in the square and diamond. *J. Supercomput.* **77**(8), 9069–9087 (2021). <https://doi.org/10.1007/s11227-020-03549-8>
9. Hoffmann, R., Désérable, D., Seredyński, F.: Cellular automata rules solving the wireless sensor network coverage problem. *Nat. Comput.* **21**, 417–447 (2022)
10. Hoffmann, R.: Generating loop patterns with a genetic algorithm and a probabilistic cellular automata rule. *Algorithms* **16**(7), 352 (2023)
11. Prusinkiewicz, P., Lindenmayer, A., Fraccica, D.: Synthesis of space-filling curves on the square grid. In: Peitgen, H.-O., Henrique, J.M., Pencdo, L.F. (eds.), *Fractals in the Fundamental and Applied Sciences*. Elsevier Science Publishers B.V., Amsterdam (1991)

12. Crossley, M.: Hamiltonian Cycles in Two Dimensional Lattices. In: Statistical Physics in Biology. Springer, Berlin/Heidelberg, Germany (2013)
13. Gardner, M.: Worm Paths. In: Knotted doughnuts and other mathematical entertainments. American Mathematical Soc. (2020)
14. Takata, D., Isokawa, T., Lee, J., Peper, F., Matsui, N.: On construction by worm-like agents on a self-timed cellular automaton. In: Cellular Automata: 10th International Conference on Cellular Automata for Research and Industry, Proceedings, vol. 10, pp. 575–84. Springer, Berlin, Heidelberg (2012)
15. Cook, M.: Still life theory. *New Constr. Cell. Automata* **226**, 93–118 (2003)